HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

INTELLIGENT SYSTEM DEVELOPMENT

BÁO CÁO BÀI TẬP 04

CONVOLUTIONAL NEURAL NETWORKS

NumPy from scratch · PyTorch · TensorFlow/Keras

Sinh viên: Nguyễn Thành Trung

Mã sinh viên: B23DCCN861

Lớp: D23CTPM01-B

Giảng viên: Dinh Que Tran, Ph.D., Assoc. Prof.

Hà Nội, 2026

MỤC LỤC

1. GIỚI THIỆU 3
2. CƠ SỞ LÝ THUYẾT CNN 3
2.1. Convolution, kernel và sliding window 3
2.2. ReLU, pooling, GlobalMaxPooling và Dense 4
2.3. Forward, loss, gradient, backpropagation và Adam 4
2.4. Tensor shapes 4
3. THIẾT KẾ THỰC NGHIỆM 5
4. CNN FROM SCRATCH BẰNG NUMPY 6
4.1. Gradient checks, parity và tiny-overfit 10
5. DIABETES 11
6. HOUSE PRICE 14
7. COMMENTS 18
8. SO SÁNH NUMPY – PYTORCH – TENSORFLOW 21
9. SO SÁNH BASIC CNN – IMPROVED CNN 21
10. THẢO LUẬN KẾT QUẢ 22
10.1. Vì sao Comments tốt nhất? 23
10.2. Vì sao Diabetes chỉ F1 khoảng 0,45? 23
10.3. Vì sao House R^2 thấp? 23
10.4. Framework gần nhau đến mức nào? 23
10.5. Scratch chứng minh điều gì? 23
11. HẠN CHẾ 23
12. KẾT LUẬN 24
13. TÀI LIỆU THAM KHẢO 25

1. GIỚI THIỆU

Assignment 04 khảo sát Convolutional Neural Network (CNN) ở ba mức triển khai nhưng cùng một nền tảng toán học: NumPy from scratch để nhìn rõ forward/backward; PyTorch với automatic differentiation; và TensorFlow/Keras với graph/layer API. Ba bài toán gồm phân loại Diabetes trên dữ liệu bảng, hồi quy House Price trên sáu field, và phân loại cảm xúc Comments theo chuỗi token. Mục tiêu là chứng minh cách CNN hoạt động, kiểm tra numerical parity và đánh giá trung thực mức phù hợp của inductive bias tích chập với từng loại dữ liệu.

Báo cáo chỉ đọc code, notebook đã execute, CSV/JSON kết quả, model và figure hiện có. Không notebook nào được chạy lại; không retrain, thay threshold, clip dự đoán để cải thiện metric, hoặc sửa output. Vì vậy các kết luận gắn với đúng split, seed và checkpoint đã khóa.

Câu hỏi nghiên cứu: (1) implementation NumPy có đúng gradient và tương đương số với framework không; (2) Improved CNN có tốt hơn Basic trên parity validation không; (3) PyTorch và TensorFlow có cho kết quả full-scale gần nhau không; (4) locality của Conv1D phù hợp khác nhau thế nào giữa tabular và text?

Bảng 1. Tóm tắt dataset và thiết kế đánh giá

Task	Nguồn/biểu diễn	Master split	Bài toán	Metric chính
Diabetes	BRFSS 2015; 253.680 dòng, 21 features	177.576 / 38.052 / 38.052	Phân loại mất cân bằng	F1, Recall, AUC
House Price	400.000 dòng lấy mẫu; 6 field × 8 chiều	280.000 / 60.000 / 60.000	Hồi quy	MAE, RMSE, R^2 (USD)
Comments	Token IDs; vocab 20.000; MAX_LEN 192	800.000 / 100.000 / 200.000	Phân loại cân bằng	F1, Accuracy, AUC

2. CƠ SỞ LÝ THUYẾT CNN

2.1. Convolution, kernel và sliding window

Trong implementation này, Conv1D thực hiện cross-correlation: $z[b,o,t] = b[o] + \Sigma c\ \Sigma j\ W[o,c,j]\cdot x[b,c,t+j-p]$. Một kernel nhỏ trượt dọc chiều dài L; mỗi output tại t chỉ nối với một vùng cục bộ của input (local connectivity). Cùng bộ weight W được tái sử dụng ở mọi vị trí (weight sharing), làm số tham số không tăng theo chiều dài chuỗi và cho phép một pattern được nhận ra dù xuất hiện ở vị trí khác.

Kernel được học, không được viết tay để tìm sẵn một motif. Trong giai đoạn đầu, filter có thể phản ứng với tương phản hoặc tổ hợp cục bộ đơn giản; ở tầng sâu hơn, filter kết hợp

activation trước thành pattern trừu tượng hơn. Cụm từ learned filters nghĩa là gradient từ loss điều chỉnh các hệ số kernel qua nhiều batch.

Ví dụ Lecture 04: Với x = [2, 1, 3, 4, 2] và k = [0,5; −1; 0,5], phép tính cửa sổ đầu là 2×0,5 + 1×(−1) + 3×0,5 = 1,5. Phép tính trực tiếp cho kết quả 1,5; implementation sử dụng kết quả toán học này, không sửa phép toán để ép thành 1,0. Ba cửa sổ valid lần lượt cho [1,5; −0,5; −1,5].

2.2. ReLU, pooling, GlobalMaxPooling và Dense

ReLU a=max(0,z) giữ activation dương và đặt phần âm về 0, tạo phi tuyến để nhiều convolution không sụp thành một biến đổi tuyến tính. MaxPool1D lấy cực đại trong cửa sổ cục bộ và giảm chiều dài; backward trả gradient về đúng vị trí argmax đã lưu. GlobalMaxPooling lấy một cực đại trên toàn bộ chiều dài cho từng channel, biến [B,C,L] thành [B,C]. Với text, mask đảm bảo PAD không được chọn làm bằng chứng. Dense sau đó tính hW^T+b để tạo hai logits classification hoặc một giá trị tuyến tính regression.

2.3. Forward, loss, gradient, backpropagation và Adam

Forward propagation đi từ input qua Conv→ReLU→Pool→GlobalMax→Dense để tạo prediction. Classification dùng softmax cross-entropy ổn định số; House dùng MSE trên target standardize(log1p(price)). Gradient là đạo hàm của loss theo mỗi tensor/tham số. Backpropagation áp dụng chain rule theo thứ tự ngược: Dense→GlobalMax→Conv3 (nếu có)→MaxPool→Conv2→Conv1, đồng thời cộng đóng góp weight sharing từ mọi cửa sổ.

Adam duy trì moment bậc một m và moment bậc hai v của gradient, hiệu chỉnh bias rồi cập nhật tham số theo $\hat{m}/(\sqrt{\hat{v}}+\varepsilon)$. NumPy thực hiện các bước này bằng ndarray; PyTorch autograd và tf.GradientTape tự dựng gradient graph. Framework thay implementation effort, không thay bản chất CNN.

2.4. Tensor shapes

Bảng 2. Kiến trúc và dòng tensor

Khối	Basic CNN	Improved CNN	Ý nghĩa
Input	[B,C,L]	[B,C,L]	NCL ở NumPy/PyTorch; TensorFlow dùng NLC
Conv1 + ReLU	[B,8,L]	[B,8,L]	8 learned filters, kernel 3, same padding
Conv2 + ReLU	[B,16,L]	[B,16,L]	16 filters dùng receptive field rộng hơn
MaxPool1D	[B,16,⌊ L/2⌋]	[B,16,⌊ L/2⌋]	kernel=2, stride=2
Conv3 + ReLU	—	[B,32,⌊ L/2⌋]	Tầng bổ sung của Improved
GlobalMaxPool	[B,16]	[B,32]	Gom chiều vị trí

Khối	Basic CNN	Improved CNN	Ý nghĩa
Hidden Dense	—	[B,16]	Biểu diễn ẩn bổ sung
Output	[B,2] hoặc [B,1]	[B,2] hoặc [B,1]	Logits hoặc linear value

3. THIẾT KẾ THỰC NGHIỆM

Parity stage chạy 2 kiến trúc × 3 implementation × 3 task = 18 runs. Mỗi task dùng cùng frozen split, initialization, objective, Adam learning rate 10^{-3}, batch order và early stopping patience=3. Validation chọn Basic/Improved; test của kiến trúc không được chọn để trống có chủ ý. Sau khi Improved thắng trung bình trên cả ba task, full-scale chỉ chạy PyTorch và TensorFlow: tổng cộng 6 runs.

Classification chọn checkpoint theo F1 validation, dùng loss làm tie-break; House chọn RMSE rồi MAE. Test chỉ được đánh giá sau selection. NumPy full-scale không chạy vì mục tiêu pedagogical đã được đáp ứng ở parity và chi phí vòng lặp thủ công lớn. Parity-test nằm trong full master-test, nên full test không phải một holdout hoàn toàn mới; không có tuning sau full test.

Bảng 3. Quy mô, cấu hình và parameter count

Task	Parity train/val/test	Full train/val/test	Batch	Basic	Improved
Diabetes	40.000 / 10.000 / 10.000	177.576 / 38.052 / 38.052	256	466	2.562
House	40.000 / 10.000 / 10.000	280.000 / 60.000 / 60.000	256	1.153	3.249
Comments	20.000 / 5.000 / 10.000	800.000 / 100.000 / 200.000	128	320.826	322.922

Bảng 4. Tóm tắt 18 parity validation runs

Task	Kiến trúc	Implementation	Params	Best epoch	Validation	Thời gian
Diabetes	Basic	scratch	466	10	F1=0.4329	212.8s
Diabetes	Basic	pytorch	466	10	F1=0.4321	60.0s
Diabetes	Basic	tensorflow	466	10	F1=0.4327	29.0s
Diabetes	Improved	scratch	2.562	7	F1=0.4526	640.7s
Diabetes	Improved	pytorch	2.562	7	F1=0.4544	95.8s
Diabetes	Improved	tensorflow	2.562	10	F1=0.4544	86.2s
House	Basic	scratch	1.153	7	RMSE=905.522 USD	165.1s
House	Basic	pytorch	1.153	7	RMSE=906.250 USD	58.2s
House	Basic	tensorflow	1.153	8	RMSE=904.995 USD	56.5s
House	Improved	scratch	3.249	4	RMSE=900.359 USD	250.8s
House	Improved	pytorch	3.249	4	RMSE=899.229 USD	25.5s
House	Improved	tensorflow	3.249	7	RMSE=908.514 USD	34.1s
Comments	Basic	scratch	320.826	4	F1=0.8691	276.5s

Task	Kiến trúc	Implementation	Params	Best epoch	Validation	Thời gian
Comments	Basic	pytorch	320.826	5	F1=0.8665	33.0s
Comments	Basic	tensorflow	320.826	5	F1=0.8650	51.5s
Comments	Improved	scratch	322.922	4	F1=0.8878	387.3s
Comments	Improved	pytorch	322.922	4	F1=0.8836	35.1s
Comments	Improved	tensorflow	322.922	5	F1=0.8868	không hợp lệ

Ghi chú timing: Comments/Improved/TensorFlow có wall time 44.830,8 giây nhưng artifact đánh dấu timing_valid=False vì timer bao gồm thời gian host/session bị suspend. Con số này không được dùng để kết luận framework nào nhanh hơn. Các timing hợp lệ khác chỉ mô tả đúng môi trường CPU/run cụ thể, không đại diện hiệu năng GPU.

4. CNN FROM SCRATCH BẰNG NUMPY

Mục tiêu: Tự cài đặt forward, backward, loss và Adam để quan sát trực tiếp sliding-window math, cache trung gian và đường truyền gradient; sau đó kiểm tra bằng finite difference và framework parity.

Đoạn mã 1. Conv1D forward — sliding window và weight sharing

```
def forward(self, x):
        x = np.asarray(x)
        if x.ndim != 3 or x.shape[1] != self.weight.shape[1]:
            raise ValueError("Conv1D input must have shape [B, C_in, L]")
        x_pad = np.pad(x, ((0, 0), (0, 0), (self.pad, self.pad)))
        windows = np.lib.stride_tricks.sliding_window_view(x_pad, self.kernel_size, axis=2)
        output = np.einsum("bclk,ock->bol", windows, self.weight) + self.bias[None, :, None]
        self._cache = (x.shape, x_pad, windows)
        return output
```

Giải thích code: sliding_window_view tạo các cửa sổ [B,C,L,K]; einsum nhân cùng weight [C_out,C_in,K] ở mọi vị trí và cộng bias. Cache giữ shape, padded input và windows cho backward.

Output thực tế

```
Input contract: [B, C_in, L]
Lecture example (valid): [1.5, -0.5, -1.5]
```

Đoạn mã 2. Conv1D backward — gradient input, kernel và bias

```
def backward(self, grad_output):
        if self._cache is None:
            raise RuntimeError("forward must be called before backward")
        original_shape, x_pad, windows = self._cache
        grad_output = np.asarray(grad_output)
        self.grad_weight[...] = np.einsum("bol,bclk->ock", grad_output, windows)
        self.grad_bias[...] = grad_output.sum(axis=(0, 2))
        grad_pad = np.zeros_like(x_pad)
        # Each kernel tap contributes to a contiguous slice of the padded
        # input gradient.  Looping over K (three taps here), rather than over
        # every sequence position, keeps the explicit sliding-window math but
        # removes Python overhead that dominates long text sequences.
        length = grad_output.shape[2]
        for tap in range(self.kernel_size):
            grad_pad[:, :, tap : tap + length] += np.einsum(
```

```
                "bol,oc->bcl", grad_output, self.weight[:, :, tap]
            )
        return grad_pad[:, :, self.pad : -self.pad] if self.pad else
grad_pad.reshape(original_shape)
```

Giải thích code: grad_weight cộng đóng góp của mọi batch/vị trí; grad_bias cộng theo batch và chiều dài. Mỗi tap của kernel phân phối upstream gradient về lát input tương ứng, đúng semantics của weight sharing.

Đoạn mã 3. ReLU và MaxPool1D

```
class ReLU:
    def forward(self, x):
        self._mask = np.asarray(x) > 0
        return np.maximum(x, 0)

    def backward(self, grad_output):
        return np.asarray(grad_output) * self._mask

    def parameters_and_grads(self):
        return []

class MaxPool1D:
    def __init__(self, kernel_size=2, stride=2):
        self.kernel_size = int(kernel_size)
        self.stride = int(stride)
        self._cache = None

    def forward(self, x):
        x = np.asarray(x)
        out_length = 1 + (x.shape[2] - self.kernel_size) // self.stride
        starts = np.arange(out_length) * self.stride
        windows = np.stack([x[:, :, start : start + self.kernel_size] for start in starts],
axis=2)
        argmax = windows.argmax(axis=3)
        self._cache = (x.shape, starts, argmax)
        return windows.max(axis=3)

    def backward(self, grad_output):
        shape, starts, argmax = self._cache
        grad_input = np.zeros(shape, dtype=np.asarray(grad_output).dtype)
        # Scatter the upstream values to the recorded maxima.  ``np.add.at``
        # preserves the accumulation semantics for overlapping pool windows,
        # while avoiding a Python loop over every batch/channel element.
        batch = np.arange(shape[0])[:, None, None]
        channel = np.arange(shape[1])[None, :, None]
        position = starts[None, None, :] + argmax
        np.add.at(grad_input, (batch, channel, position), np.asarray(grad_output))
        return grad_input

    def downsample_mask(self, mask):
        mask = np.asarray(mask, dtype=bool)
        out_length = 1 + (mask.shape[1] - self.kernel_size) // self.stride
        return np.stack(
            [mask[:, i * self.stride : i * self.stride + self.kernel_size].any(axis=1) for i
in range(out_length)],
            axis=1,
        )

    def parameters_and_grads(self):
        return []
```

Giải thích code: ReLU lưu mask z>0. MaxPool lưu starts và argmax; np.add.at scatter gradient về cực đại, kể cả khi các cửa sổ pool chồng lấn.

Đoạn mã 4. GlobalMaxPool1D có mask

```
class GlobalMaxPool1D:
    def forward(self, x, mask=None):
        x = np.asarray(x)
        valid = np.ones((x.shape[0], x.shape[2]), dtype=bool) if mask is None else
np.asarray(mask, dtype=bool)
        masked = np.where(valid[:, None, :], x, -np.inf)
        self._argmax = masked.argmax(axis=2)
        self._valid_rows = valid.any(axis=1)
        output = masked.max(axis=2)
        output[~self._valid_rows] = 0.0
        self._input_shape = x.shape
        return output

    def backward(self, grad_output):
        grad_input = np.zeros(self._input_shape, dtype=np.asarray(grad_output).dtype)
        batch = np.arange(self._input_shape[0])[:, None]
        channel = np.arange(self._input_shape[1])[None, :]
        values = np.asarray(grad_output) * self._valid_rows[:, None]
        np.add.at(grad_input, (batch, channel, self._argmax), values)
        return grad_input

    def parameters_and_grads(self):
        return []
```

Giải thích code: Vị trí invalid nhận $-\infty$ trước phép max; dòng hoàn toàn rỗng được trả 0. Backward chỉ đưa gradient tới argmax hợp lệ. Đây là cơ chế quan trọng cho PAD của Comments.

Đoạn mã 5. Dense forward/backward

```
class Dense:
    def __init__(self, in_features, out_features, *, rng=None, dtype=np.float32):
        generator = rng or np.random.default_rng(42)
        scale = np.sqrt(2.0 / in_features)
        self.weight = generator.normal(0.0, scale, (out_features,
in_features)).astype(dtype)
        self.bias = np.zeros(out_features, dtype=dtype)
        self.grad_weight = np.zeros_like(self.weight)
        self.grad_bias = np.zeros_like(self.bias)
        self._input = None

    def forward(self, x):
        self._input = np.asarray(x)
        return self._input @ self.weight.T + self.bias

    def backward(self, grad_output):
        grad_output = np.asarray(grad_output)
        self.grad_weight[...] = grad_output.T @ self._input
        self.grad_bias[...] = grad_output.sum(axis=0)
        return grad_output @ self.weight

    def parameters_and_grads(self):
        return [(self.weight, self.grad_weight), (self.bias, self.grad_bias)]
```

Giải thích code: Forward là phép affine. Backward tính $dW = dY^T X$, $db = \Sigma dY$ và $dX = dY W$, hoàn toàn bằng NumPy.

Đoạn mã 6. Loss classification và regression

```
class SoftmaxCrossEntropyLoss:
    def __init__(self, class_weights=None):
        self.class_weights = None if class_weights is None else np.asarray(class_weights)

    def forward(self, logits, targets):
        logits = np.asarray(logits)
        self.targets = np.asarray(targets, dtype=np.int64)
        shifted = logits - logits.max(axis=1, keepdims=True)
```

```
        exp = np.exp(shifted)
        self.probabilities = exp / exp.sum(axis=1, keepdims=True)
        self.sample_weights = np.ones(len(targets), dtype=logits.dtype)
        if self.class_weights is not None:
            self.sample_weights = self.class_weights[self.targets].astype(logits.dtype)
        self.normalizer = self.sample_weights.sum()
        # Compute NLL directly from shifted logits.  Taking log of an already
        # underflowed softmax probability would clip extreme losses and diverge
        # from the stable cross-entropy used by PyTorch/TensorFlow.
        log_partition = np.log(exp.sum(axis=1))
        negative_log_likelihood = -shifted[np.arange(len(targets)), self.targets] +
log_partition
        return float(np.sum(negative_log_likelihood * self.sample_weights) /
self.normalizer)

    def backward(self):
        gradient = self.probabilities.copy()
        gradient[np.arange(len(self.targets)), self.targets] -= 1.0
        gradient *= self.sample_weights[:, None] / self.normalizer
        return gradient

class MSELoss:
    def forward(self, predictions, targets):
        self.difference = np.asarray(predictions) - np.asarray(targets)
        return float(np.mean(self.difference ** 2))

    def backward(self):
        return 2.0 * self.difference / self.difference.size
```

Giải thích code: Cross-entropy trừ max logit để ổn định, hỗ trợ class weight và trả gradient chuẩn hóa. MSE trả 2(pred−target)/N.

Đoạn mã 7. Manual Adam

```
class Adam:
    def __init__(self, parameters_and_grads, learning_rate=1e-3, beta1=0.9, beta2=0.999,
epsilon=1e-7):
        self.items = list(parameters_and_grads)
        self.learning_rate = learning_rate
        self.beta1, self.beta2, self.epsilon = beta1, beta2, epsilon
        self.m = [np.zeros_like(parameter) for parameter, _ in self.items]
        self.v = [np.zeros_like(parameter) for parameter, _ in self.items]
        self.step_number = 0

    def step(self):
        self.step_number += 1
        for index, (parameter, gradient) in enumerate(self.items):
            self.m[index] = self.beta1 * self.m[index] + (1 - self.beta1) * gradient
            self.v[index] = self.beta2 * self.v[index] + (1 - self.beta2) * gradient *
gradient
            m_hat = self.m[index] / (1 - self.beta1 ** self.step_number)
            v_hat = self.v[index] / (1 - self.beta2 ** self.step_number)
            parameter -= self.learning_rate * m_hat / (np.sqrt(v_hat) + self.epsilon)
```

Giải thích code: Mỗi parameter có m, v riêng; bias correction dùng step_number. Update được thực hiện in-place trên ndarray, không gọi optimizer của framework.

Đoạn mã 8. Forward/backward toàn mạng ScratchCNN

```
def forward(self, x, mask=None):
        value = self.relu1.forward(self.conv1.forward(x))
        value = self.relu2.forward(self.conv2.forward(value))
        value = self.pool.forward(value)
        self._pooled_mask = None if mask is None else self.pool.downsample_mask(mask)
        if self.improved:
            value = self.relu3.forward(self.conv3.forward(value))
        value = self.global_pool.forward(value, self._pooled_mask)
        if self.improved:
```

```
            value = self.relu_hidden.forward(self.hidden.forward(value))
        return self.output.forward(value)

def backward(self, grad_logits):
        gradient = self.output.backward(grad_logits)
        if self.improved:
            gradient = self.hidden.backward(self.relu_hidden.backward(gradient))
        gradient = self.global_pool.backward(gradient)
        if self.improved:
            gradient = self.conv3.backward(self.relu3.backward(gradient))
        gradient = self.pool.backward(gradient)
        gradient = self.conv2.backward(self.relu2.backward(gradient))
        return self.conv1.backward(self.relu1.backward(gradient))
```

Giải thích code: Backward đi ngược chính xác thứ tự forward. Basic bỏ conv3/hidden; Improved thêm hai phép biến đổi trainable nhưng dùng cùng loss và protocol.

4.1. Gradient checks, parity và tiny-overfit

Đoạn mã 9. Finite-difference và numerical parity

```
def relative_error(actual, expected):
    actual, expected = np.asarray(actual), np.asarray(expected)
    denominator = np.maximum(1e-12, np.abs(actual) + np.abs(expected))
    return float(np.max(np.abs(actual - expected) / denominator))

# finite_difference_checks(...) perturbs each element by ±epsilon
# forward_parity(...) copies canonical NumPy weights to Torch/TF
```

Giải thích code: Finite difference kiểm tra đạo hàm giải tích mà không phụ thuộc autograd. Sai số 10^{-13}–10^{-10} và backward parity ~10^{-16} là bằng chứng mạnh rằng công thức gradient đúng. Forward sai khác tối đa cỡ 10^{-7} phù hợp round-off float32.

Output thực tế

```
Finite-difference relative errors
conv_db: 7.111e-11  PASS
conv_dw: 1.791e-10  PASS
conv_dx: 4.106e-10  PASS
cross_entropy: 3.922e-11  PASS
dense_db: 9.754e-12  PASS
dense_dw: 9.457e-11  PASS
dense_dx: 2.667e-10  PASS
embedding: 7.266e-13  PASS
mse: 3.044e-10  PASS

Conv1D backward ↔ PyTorch
db: 3.500e-16  PASS
dw: 4.409e-16  PASS
dx: 2.978e-16  PASS

Base forward parity (max abs)
Basic scratch↔torch logits: 7.153e-07
Basic scratch↔TF logits: 4.768e-07
Improved scratch↔torch logits: 2.384e-07
Improved scratch↔TF logits: 4.768e-07
PASS
```

Output kiểm chứng đã lưu

```
comments/basic: scratch↔PT=2.980e-08; scratch↔TF=4.470e-08  PASS
comments/improved: scratch↔PT=1.043e-07; scratch↔TF=1.043e-07  PASS
diabetes/basic: scratch↔PT=2.384e-07; scratch↔TF=4.768e-07  PASS
diabetes/improved: scratch↔PT=4.768e-07; scratch↔TF=7.153e-07  PASS
house/basic: scratch↔PT=6.519e-09; scratch↔TF=7.451e-09  PASS
house/improved: scratch↔PT=2.980e-08; scratch↔TF=2.980e-08  PASS
tiny-overfit NumPy: 6/6 cấu hình giảm loss theo ngưỡng test (<80% initial)  PASS
```

```
tiny-overfit PyTorch/TensorFlow: 4/4 cấu hình giảm loss (<95% initial)  PASS
pytest artifact: 57 passed in 49.76s
```

Artifact không lưu từng giá trị initial/final loss của tiny-overfit, vì vậy báo cáo chỉ ghi trạng thái PASS và tiêu chí assertion thực tế từ test, không bịa thêm loss. Các test này chứng minh đường optimizer/backprop có thể học một batch nhỏ; chúng không phải metric generalization.

5. DIABETES

Mục tiêu: Phân loại nguy cơ diabetes trên BRFSS 2015, đồng thời đánh giá hệ quả của class imbalance và tính không tự nhiên của locality trên feature tabular.

Dataset có 253.680 dòng và 21 features. Lecture minh họa một vector 8 features, nhưng project dùng đúng dataset 21-feature đã được duy trì xuyên suốt Assignment 04. Bảy feature liên tục/ordinal được standardize bằng thống kê master-train; các feature nhị phân được passthrough. Output mỗi dòng là [1,21], rồi NCL batch là [B,1,21].

Train có 24.742 positive trên 177.576 mẫu (13,93%); validation/test mỗi tập có 5.302 positive trên 38.052 (13,93%). Class weight trong training làm model ưu tiên phát hiện positive. Vì 21 feature chỉ được sắp theo một thứ tự semantic do người thiết kế chọn, láng giềng trong chuỗi không có locality tự nhiên như pixel hoặc token; CNN ở đây là teaching experiment, không phải tuyên bố kiến trúc tối ưu cho tabular.

Đoạn mã 10. Diabetes CNN thực tế (PyTorch và TensorFlow; input_channels=1, output_dim=2)

```
class TorchCNN(nn.Module):
    def __init__(self, input_channels, output_dim, *, improved=False):
        super().__init__()
        self.improved = improved
        self.conv1 = nn.Conv1d(input_channels, 8, 3, padding=1)
        self.conv2 = nn.Conv1d(8, 16, 3, padding=1)
        self.pool = nn.MaxPool1d(2, 2)
        if improved:
            self.conv3 = nn.Conv1d(16, 32, 3, padding=1)
            self.hidden = nn.Linear(32, 16)
            self.output = nn.Linear(16, output_dim)
        else:
            self.output = nn.Linear(16, output_dim)

    def forward(self, x, mask=None):
        x = torch.relu(self.conv1(x))
        x = torch.relu(self.conv2(x))
        x = self.pool(x)
        pooled_mask = None if mask is None else _pool_mask(mask)
        if self.improved:
            x = torch.relu(self.conv3(x))
        x = _masked_global_max(x, pooled_mask)
        if self.improved:
            x = torch.relu(self.hidden(x))
        return self.output(x)

    @property
    def trainable_layers(self):
        return 5 if self.improved else 3

class TFCNN(tf.keras.Model):
```

```
    def __init__(self, input_channels, output_dim, *, improved=False):
        super().__init__()
        self.improved = improved
        self.conv1 = tf.keras.layers.Conv1D(8, 3, padding="same", name="conv1")
        self.conv2 = tf.keras.layers.Conv1D(16, 3, padding="same", name="conv2")
        self.pool = tf.keras.layers.MaxPooling1D(2, 2, name="pool")
        if improved:
            self.conv3 = tf.keras.layers.Conv1D(32, 3, padding="same", name="conv3")
            self.hidden = tf.keras.layers.Dense(16, name="hidden")
        self.output_layer = tf.keras.layers.Dense(output_dim, name="output")

    def call(self, x, mask=None):
        x = tf.nn.relu(self.conv1(x))
        x = tf.nn.relu(self.conv2(x))
        x = self.pool(x)
        pooled_mask = None
        if mask is not None:
            usable = (tf.shape(mask)[1] // 2) * 2
            pooled_mask = tf.reduce_any(tf.reshape(mask[:, :usable], (tf.shape(mask)[0], -1,
2)), axis=2)
        if self.improved:
            x = tf.nn.relu(self.conv3(x))
        if pooled_mask is None:
            x = tf.reduce_max(x, axis=1)
        else:
            minimum = tf.cast(-1e30, x.dtype)
            x = tf.reduce_max(tf.where(pooled_mask[:, :, None], x, minimum), axis=1)
            x = tf.where(tf.reduce_any(pooled_mask, axis=1, keepdims=True), x,
tf.zeros_like(x))
        if self.improved:
            x = tf.nn.relu(self.hidden(x))
        return self.output_layer(x)

    @property
    def trainable_layer_count(self):
        return 5 if self.improved else 3
```

Giải thích code: Đây là class thật được instantiate cho Diabetes; không phải generic pseudo-code. TensorFlow dùng cùng topology ở layout NLC, còn ScratchCNN dùng NCL.

Output thực tế

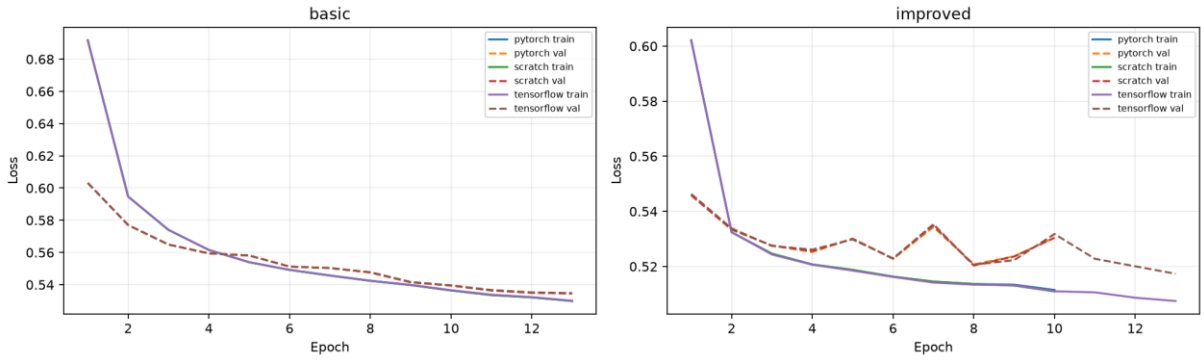
```
Input [B,1,21] → Conv1 [B,8,21] → Conv2 [B,16,21] → Pool [B,16,10] → Conv3 [B,32,10] →
GlobalMax [B,32] → Dense [B,16] → logits [B,2]
Improved parameters: 2,562
```

Bảng 5. Diabetes full-scale test

Framework	Accuracy	Precision	Recall	F1	ROC-AUC	Confusion matrix [[TN,FP],[FN,TP]]
Pytorch	0.7545	0.3287	0.7309	0.4535	0.8241	[[24836, 7914], [1427, 3875]]
Tensorflow	0.7608	0.3322	0.7092	0.4524	0.8219	[[25191, 7559], [1542, 3760]]

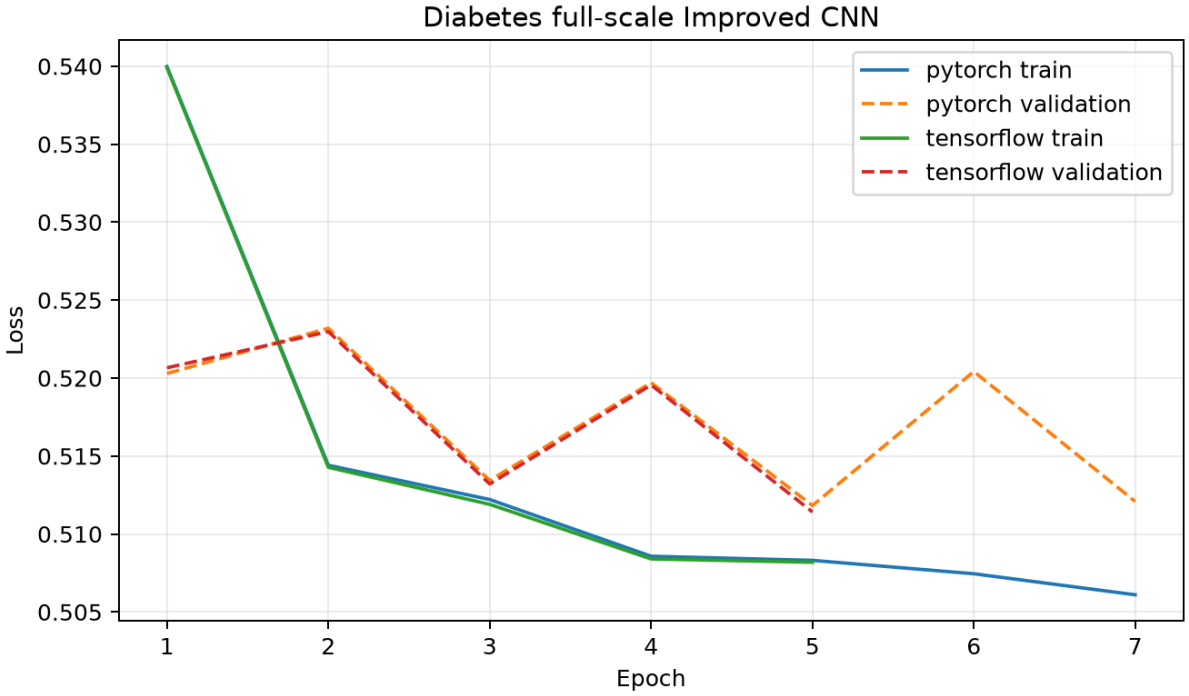
Phân tích output: PyTorch đạt Accuracy 0,7545, Precision 0,3287, Recall 0,7309, F1 0,4535, AUC 0,8241; TensorFlow tương ứng 0,7608 / 0,3322 / 0,7092 / 0,4524 / 0,8219. Accuracy cao một phần do negative chiếm khoảng 86%; vì vậy F1, Recall và AUC phải được đọc cùng nhau. Class weighting dịch quyết định về phía Recall: phát hiện 3.875/5.302 positive ở PyTorch nhưng tạo 7.914 false positive. TensorFlow giảm FP xuống 7.559, đổi

lại FN tăng từ 1.427 lên 1.542. Đây là precision–recall trade-off chứ không phải framework nào tuyệt đối tốt hơn.



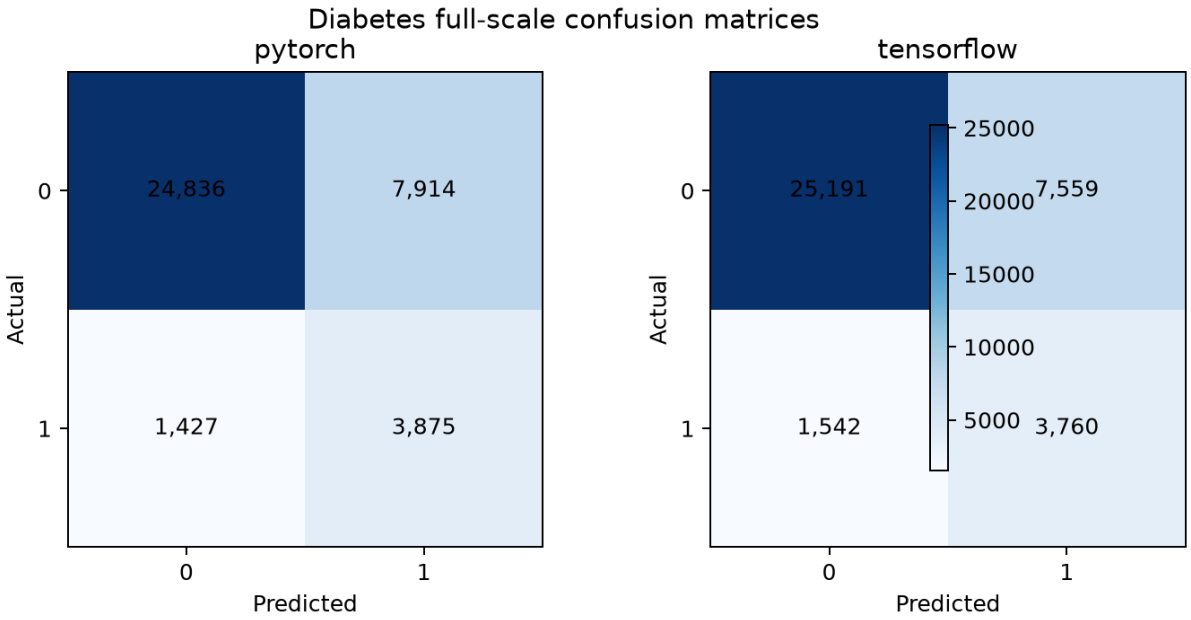
Hình 1. Diabetes — loss curves của 6 parity runs

Phân tích hình: Các đường loss cho thấy cả ba implementation học được cùng xu hướng; mức lệch sau nhiều update có thể xuất hiện do kernel/round-off/optimizer, dù initial forward đã parity. Figure hỗ trợ việc đọc validation, không thay thế bảng metric.



Hình 2. Diabetes — training/validation loss full-scale Improved

Phân tích hình: Loss train giảm nhẹ liên tục, trong khi validation dao động theo trade-off class-weighted objective. Best epoch là 4 cho PyTorch và 2 cho TensorFlow theo F1 rồi loss; checkpoint tốt nhất, không phải epoch cuối, được dùng cho test.



Hình 3. Diabetes — confusion matrices full-scale

Phân tích hình: Hai ma trận rất gần nhau và cùng bộc lộ nhiều false positive. PyTorch giữ Recall cao hơn; TensorFlow có Accuracy/Precision nhỉnh hơn. Với sàng lọc, Recall có thể quan trọng, nhưng precision thấp đòi hỏi bước xác nhận tiếp theo và tuyệt đối không biến model thành công cụ chẩn đoán.

Kết luận task: CNN học được tín hiệu phân biệt (AUC ≈0,82) nhưng F1 chỉ ≈0,45 do lớp dương hiếm, threshold argmax và locality feature mang tính quy ước. Kết quả phù hợp vai trò thí nghiệm giảng dạy hơn là bằng chứng ưu thế CNN trên tabular.

6. HOUSE PRICE

Mục tiêu: Hồi quy giá nhà với split không rò rỉ property group, biểu diễn sáu field thành sáu vị trí và trả metric về USD.

Canonical loader lọc 2.226.382 dòng raw, có 539.432 dòng hợp lệ và lấy mẫu 400.000 dòng với seed 42. Group-aware split tạo 392.384 property groups, group lớn nhất 19 dòng và zero overlap giữa train/validation/test. Sáu vị trí theo thứ tự state, status, acre_lot, house_size, bed, bath. Hai category dùng embedding riêng; bốn field số được chiếu từng field thành vector 8 chiều bằng weight/bias học được. Kết quả encoder [B,6,8] được transpose thành [B,8,6] cho NumPy/PyTorch.

Target được biến đổi standardize(log1p(price)) bằng mean/scale chỉ học trên master-train; output CNN là một giá trị tuyến tính không bị ràng buộc. Khi đánh giá, output được inverse bằng expm1 và MAE/RMSE/R^2 được tính trên USD. CNN trên sáu vị trí field vẫn là teaching representation: thứ tự field không tạo translation invariance tự nhiên như ảnh/text.

Đoạn mã 11. HouseCNN + FieldEncoder thực tế (NumPy)

```
def forward(self, numeric, state_ids, status_ids):
        self._numeric = np.asarray(numeric)
        self._state_ids = np.asarray(state_ids)[:, None]
        self._status_ids = np.asarray(status_ids)[:, None]
        numeric_tokens = self._numeric[:, :, None] * self.numeric_weight[None, :, :] +
self.numeric_bias[None, :, :]
        state = self.state_embedding.forward(self._state_ids)
        status = self.status_embedding.forward(self._status_ids)
        return np.concatenate([state, status, numeric_tokens], axis=1).transpose(0, 2, 1)

class ScratchHouseCNN:
    def __init__(self, state_size, status_size, *, improved=False, rng=None,
dtype=np.float32):
        generator = rng or np.random.default_rng(42)
        self.encoder = ScratchHouseFieldEncoder(state_size, status_size, rng=generator,
dtype=dtype)
        self.cnn = ScratchCNN(8, 1, improved=improved, rng=generator, dtype=dtype)

    def forward(self, numeric, state_ids, status_ids):
        return self.cnn.forward(self.encoder.forward(numeric, state_ids, status_ids))

    def backward(self, grad_output):
        return self.encoder.backward(self.cnn.backward(grad_output))

    def parameters_and_grads(self):
        return self.encoder.parameters_and_grads() + self.cnn.parameters_and_grads()
```

Giải thích code: Mỗi numeric field có phép affine độc lập sang 8 chiều; state/status giữ vocabulary riêng. ID 0 ở House nghĩa missing/OOV và vẫn có learned vector, khác PAD=0 của Comments.

Output thực tế

```
numeric [B,4], state/status [B] → field tokens [B,6,8] → CNN input [B,8,6] → linear output
[B,1]
Field encoder parameters: 536; Improved total: 3,249
```

Đoạn mã 12. Inverse log-target và xử lý candidate không hữu hạn

```
def _inverse_target_from_metadata(metadata):
    mean, scale = float(metadata["target_mean"]), float(metadata["target_scale"])
    # USD reporting must use float64: a finite standardized prediction can
    # legitimately exceed float32's expm1 range on extreme House rows.
    return lambda values: np.expm1(np.asarray(values, dtype=np.float64).reshape(-1) * scale
+ mean)

def regression_metrics(y_true, prediction):
    y_true = np.asarray(y_true, dtype=np.float64)
    prediction = np.asarray(prediction, dtype=np.float64)
    # A transient House checkpoint can exponentiate beyond float64 on an
    # extreme row. It is scientifically the worst possible validation
    # candidate, not a reason to abort before restoring the best checkpoint.
    if not np.isfinite(y_true).all() or not np.isfinite(prediction).all():
        return {"mae": float("inf"), "rmse": float("inf"), "r2": float("-inf")}
    mse = mean_squared_error(y_true, prediction)
    return {
        "mae": float(mean_absolute_error(y_true, prediction)),
        "rmse": float(np.sqrt(mse)),
        "r2": float(r2_score(y_true, prediction)),
    }
```

Giải thích code: Code chuyển prediction sang float64 trước expm1. Nếu extreme prediction overflow, evaluator trả inf/−inf để candidate trở thành tệ nhất thay vì crash. Không có clipping prediction để cải thiện metric; best checkpoint hữu hạn được phục hồi sau early stopping.

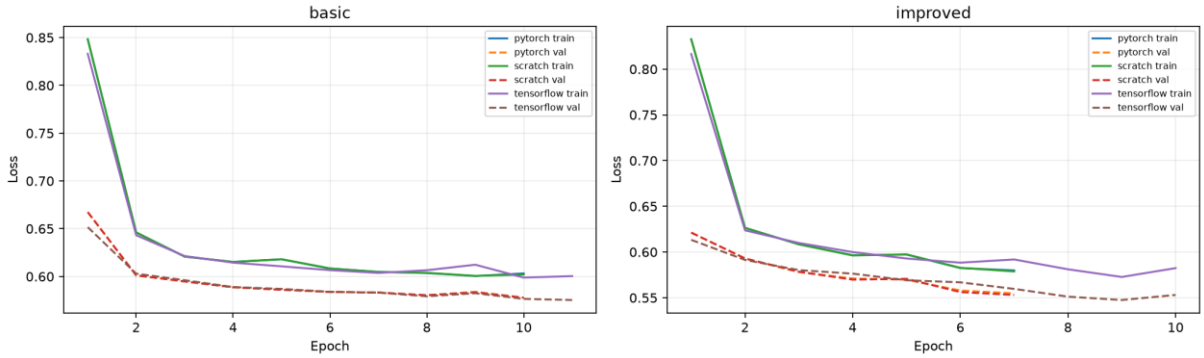
Output thực tế

```
Intermediate history: PyTorch epochs 8,9,11 có RMSE=inf; TensorFlow epochs 3,9,13 có
RMSE=inf.
Best checkpoints: epoch 10 cho cả hai; final test metrics đều hữu hạn.
```

Bảng 6. House Price full-scale test (đã inverse về USD)

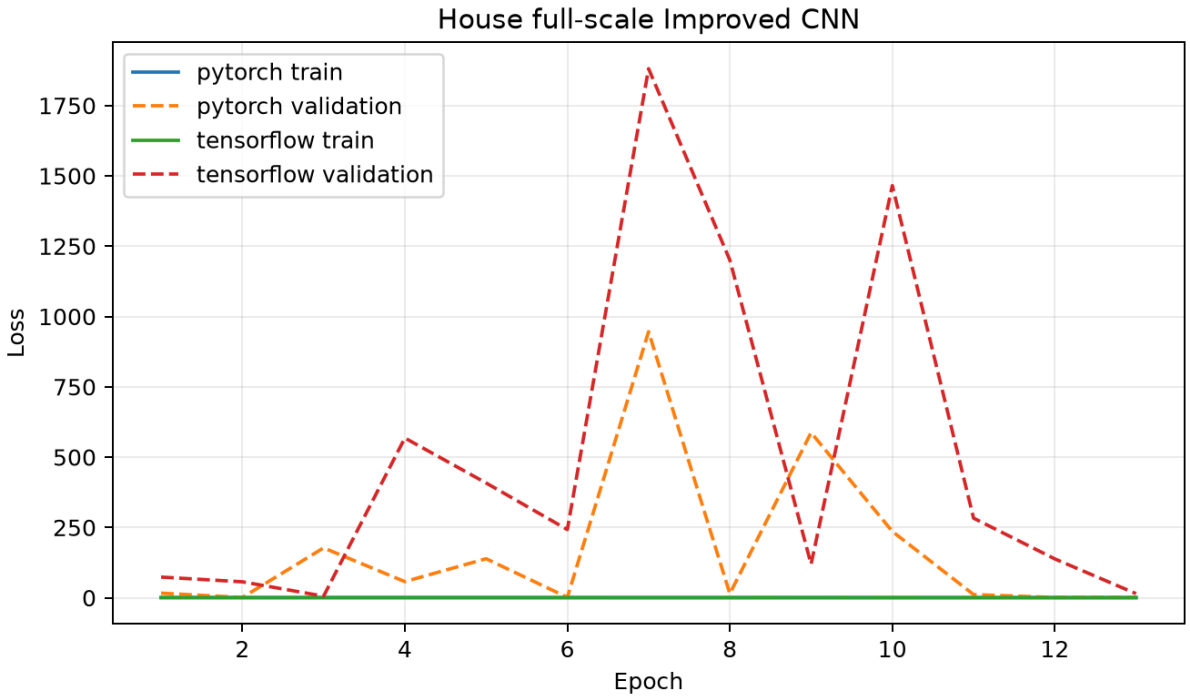
Framework	Best epoch	MAE	RMSE	R^2	Trạng thái
Pytorch	10	252.719 USD	1.290.996 USD	0.1737	Hữu hạn
Tensorflow	10	255.106 USD	1.291.741 USD	0.1727	Hữu hạn

Phân tích output: PyTorch đạt MAE ≈252.719 USD, RMSE ≈1.290.996 USD, R^2 ≈0,1737; TensorFlow đạt ≈255.106 USD / ≈1.291.741 USD / ≈0,1727. R^2 thấp được giữ nguyên: model chỉ giải thích khoảng 17,3% phương sai test so với baseline trung bình. RMSE gấp khoảng 5 lần MAE vì squared error bị chi phối bởi một số tài sản cực đắt/sai số cực lớn; test max lên tới 212,5 triệu USD. Log1p giúp training nhưng không xóa heavy tail sau inverse.



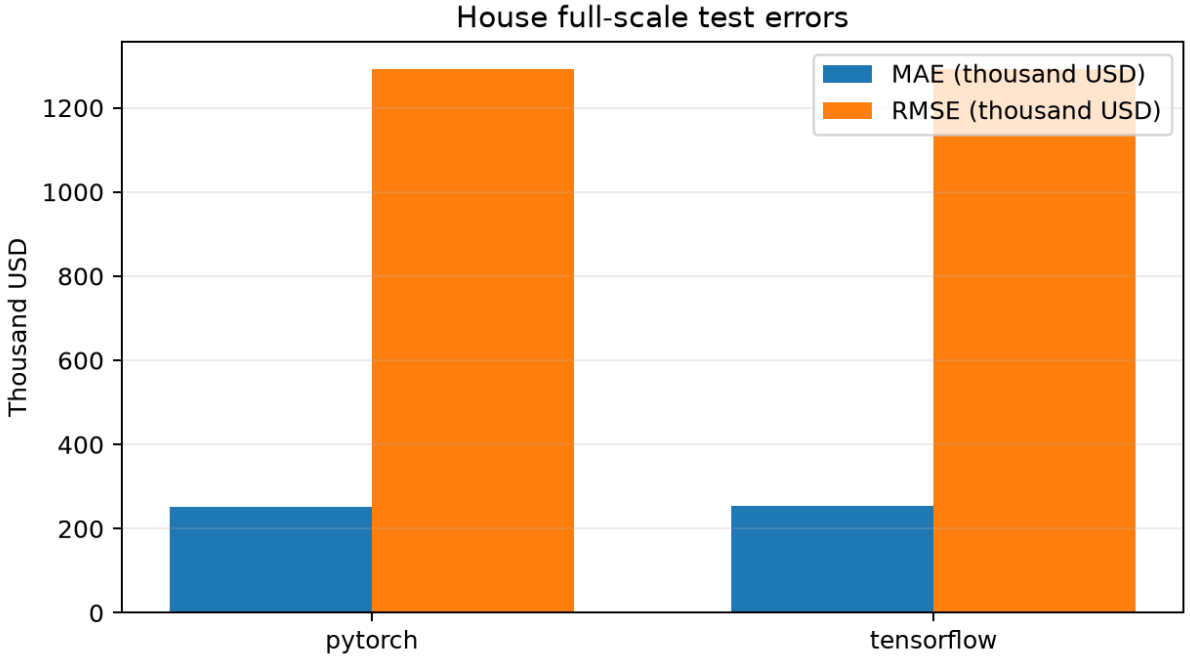
Hình 4. House Price — loss curves của 6 parity runs

Phân tích hình: Cả Basic và Improved giảm loss trên standardized log-target, nhưng validation RMSE USD có thể nhạy hơn nhiều khi inverse. Vì vậy selection dùng RMSE/MAE USD, không chỉ nhìn MSE transformed.



Hình 5. House Price — training/validation loss full-scale Improved

Phân tích hình: Validation loss có spike lớn dù train loss tương đối nhỏ, phản ánh một số extreme rows. Figure được đọc cùng history: các candidate overflow bị loại; epoch 10 là checkpoint tốt nhất hữu hạn ở cả hai framework.



Hình 6. House Price — MAE và RMSE full-scale

Phân tích hình: Hai framework gần như trùng RMSE; PyTorch nhỉnh hơn nhẹ ở cả MAE và RMSE. Khoảng cách lớn giữa MAE và RMSE trực quan hóa tác động heavy tail/outlier, không nên bị che bằng một nhận xét chung rằng model ‘tốt’.

Kết luận task: CNN không mạnh trên House. Sáu field thiếu location/time/quality chi tiết, locality nhân tạo và target đuôi rất nặng khiến R^2 thấp. Kết quả phù hợp như bài tập biểu diễn và kiểm soát numerical failure, chưa phù hợp định giá chuyên nghiệp.

7. COMMENTS

Mục tiêu: Giữ thứ tự từ để Conv1D học local token patterns, thay cho pipeline TF-IDF→MLP của Assignment 03.

Assignment 03 dùng TF-IDF → MLP: vector thưa không giữ vị trí tuyệt đối của token. Assignment 04 dùng token IDs → Embedding → Conv1D → Pooling → Dense. Vocabulary tối đa 20.000 gồm PAD=0 và OOV=1; embedding 16 chiều; MAX_LEN=192. Chuỗi được chuẩn hóa NFC, lowercase và collapse whitespace. Vocabulary và thống kê độ dài chỉ fit trên 800.000 train samples.

Bảng 7. Train-only token statistics của Comments

Count	Min	p50	p75	p90	p95	p99	Max	Truncation@192
800.000	2	71.0	109.0	146.0	163.0	184.0	257	0.2697%

p95=163 nên 192 là candidate nhỏ nhất bao phủ trên p95; truncation train chỉ 0,26975%. Validation OOV token rate 2,8963%, test 2,9048%; truncation tương ứng 0,2650% và 0,2515%. Duplicate overlap được loại về 0 giữa các split trước huấn luyện.

Đoạn mã 13. TextCNN + Embedding/PAD masking thực tế

```
class ScratchTextCNN:
    def __init__(self, vocabulary_size, output_dim=2, *, improved=False, rng=None,
dtype=np.float32):
        generator = rng or np.random.default_rng(42)
        self.embedding = Embedding(vocabulary_size, 16, padding_idx=0, rng=generator,
dtype=dtype)
        self.cnn = ScratchCNN(16, output_dim, improved=improved, rng=generator, dtype=dtype)

    def forward(self, token_ids, mask=None):
        embedded = self.embedding.forward(token_ids).transpose(0, 2, 1)
        return self.cnn.forward(embedded, token_ids != 0 if mask is None else mask)

    def backward(self, grad_logits):
        grad_embedding = self.cnn.backward(grad_logits).transpose(0, 2, 1)
        self.embedding.backward(grad_embedding)

    def parameters_and_grads(self):
        return self.embedding.parameters_and_grads() + self.cnn.parameters_and_grads()

def embed_tokens(self, ids):
        ids = tf.convert_to_tensor(ids)
        embedded = self.embedding(ids)
        # Zero PAD values in the computation graph; its embedding row receives
        # no gradient and cannot create artificial convolution activations.
        return tf.where(tf.not_equal(ids, 0)[:, :, None], embedded, tf.zeros_like(embedded))

def call(self, ids):
        return self.cnn(self.embed_tokens(ids), tf.not_equal(ids, 0))
```

Giải thích code: Scratch/PyTorch transpose embedding sang NCL. TensorFlow giữ NLC và zero PAD ngay trong graph; explicit mask được truyền qua pooling/GlobalMax. PAD không tạo activation giả và hàng embedding PAD không nhận gradient.

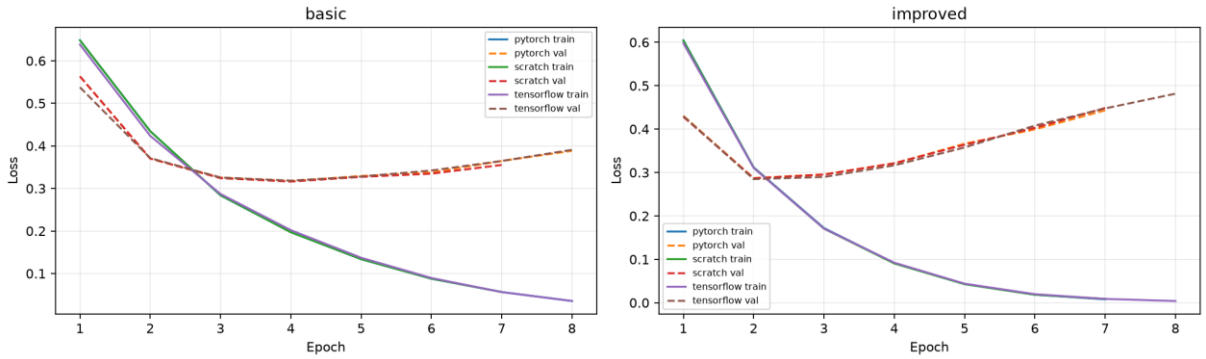
Output thực tế

```
token_ids [B,192] → embedding [B,192,16] → transpose/NLC Conv1D → pooled channels → logits
[B,2]
Improved parameters: 322,922
```

Bảng 8. Comments full-scale test

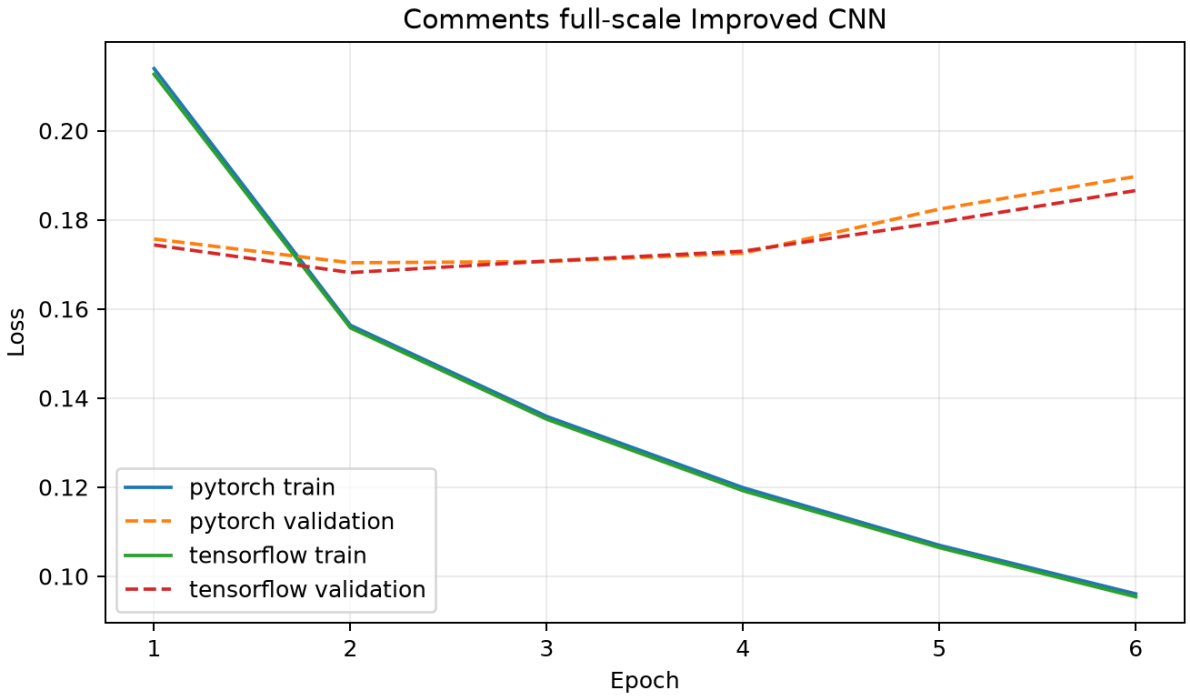
Framework	Accuracy	Precision	Recall	F1	ROC-AUC	Confusion matrix [[TN,FP],[FN,TP]]
Pytorch	0.9359	0.9299	0.9428	0.9363	0.9823	[[92894, 7106], [5720, 94280]]
Tensorflow	0.9362	0.9286	0.9450	0.9367	0.9823	[[92730, 7270], [5496, 94504]]

Phân tích output: PyTorch đạt Accuracy 0,9359, Precision 0,9299, Recall 0,9428, F1 0,9363, AUC 0,9823; TensorFlow đạt 0,9362 / 0,9286 / 0,9450 / 0,9367 / 0,9823. Chênh lệch F1 chỉ khoảng 0,00042 và AUC khoảng 0,00004; hai implementation thực chất hội tụ đến chất lượng gần tương đương. Comments tốt nhất vì cửa sổ token gần nhau mang meaning: n-gram sentiment, phủ định và cụm từ đánh giá có thể được một filter nhận ra ở nhiều vị trí nhờ weight sharing.



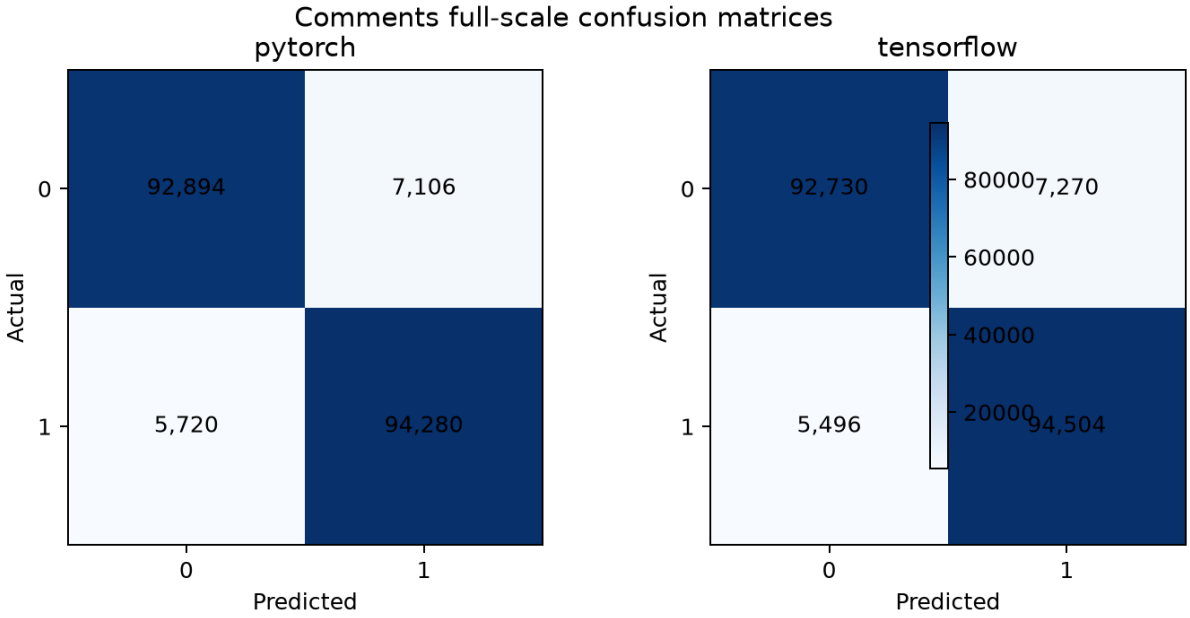
Hình 7. Comments — loss curves của 6 parity runs

Phân tích hình: Improved đạt validation F1 trung bình cao hơn Basic dù validation loss trung bình không thấp hơn. Điều này nhắc rằng selection metric là F1; loss và F1 có thể phản ánh các khía cạnh khác nhau của confidence/decision boundary.



Hình 8. Comments — training/validation loss full-scale Improved

Phân tích hình: Train loss tiếp tục giảm từ khoảng 0,214 xuống 0,096, trong khi validation loss tốt nhất quanh epoch 2 rồi tăng dần đến khoảng 0,19/0,187 ở epoch 6. Đây là dấu hiệu overfitting sau vài epoch; checkpoint theo F1 chọn epoch 3, trước khi gap mở rộng mạnh.



Hình 9. Comments — confusion matrices full-scale

Phân tích hình: Hai matrix cân bằng và gần nhau. TensorFlow giảm false negative (5.496 so với 5.720) nhưng tăng false positive (7.270 so với 7.106), giải thích Recall cao hơn và Precision thấp hơn nhẹ.

Kết luận task: Comments là task phù hợp nhất với Conv1D trong ba bài toán vì locality của token có ý nghĩa. Kết quả cao không loại bỏ giới hạn vocabulary hữu hạn, OOV, truncation, sarcasm/ngữ cảnh dài và embedding học từ đầu.

8. SO SÁNH NUMPY – PYTORCH – TENSORFLOW

Ba implementation thực hiện cùng phép toán và dùng canonical initialization được chuyển đúng layout. NumPy/PyTorch dùng NCL; TensorFlow dùng NLC, nên kernel Conv1D được transpose khi copy. Initial forward parity trên cả task/architecture có max absolute difference từ $6{,}5\times10^{-9}$ đến $7{,}15\times10^{-7}$. Điều này cho thấy framework thay cách biểu diễn và tự động hóa, không thay ý tưởng toán học.

Bảng 9. So sánh ba implementation

Tiêu chí	NumPy from scratch	PyTorch	TensorFlow/Keras
Mathematics	Tường minh window/einsum/chain rule	Cùng Conv/ReLU/Pool/Dense	Cùng Conv/ReLU/Pool/Dense
Implementation effort	Cao; tự cache và viết gradient	Gọn; nn.Module	Gọn; Layer/Model API
Automatic differentiation	Không; backward thủ công	autograd + loss.backward()	tf.GradientTape
Numerical parity	Nguồn canonical	Max abs ban đầu $\leq7{,}15\times10^{-7}$	Max abs ban đầu $\leq7{,}15\times10^{-7}$
Runtime	Chậm hơn trong các timing hợp lệ parity	Tối ưu backend	Tối ưu backend; 1 timing parity invalid bị loại
Usability	Tốt để học/kiểm chứng	Debug linh hoạt, ecosystem mạnh	High-level API, graph/deployment thuận tiện
Scalability	Hạn chế do memory/window và Python orchestration	Batch/GPU/distributed	Batch/GPU/distributed

Không dùng timing invalid để xếp hạng. Ở full-scale CPU, TensorFlow ngắn hơn PyTorch trong cả ba run đã lưu (60,1<109,4 giây; 220,3<309,7; 1.012,3<1.288,9), nhưng đây chỉ là quan sát trên một máy, cấu hình và implementation cụ thể. Chất lượng metric gần nhau hơn nhiều so với khác biệt runtime.

Kết luận: Scratch chứng minh CNN mathematics và tạo oracle kiểm chứng; framework giải phóng công sức khỏi gradient bookkeeping để mở rộng dataset/hardware. Sự tiện dụng và tốc độ không biến một inductive bias không phù hợp thành phù hợp.

9. SO SÁNH BASIC CNN – IMPROVED CNN

Basic có Conv1→Conv2→Pool→GlobalMax→Output (3 phép biến đổi trainable). Improved thêm Conv3 16→32 và hidden Dense 32→16 (5 phép biến đổi). Mọi yếu tố khác trong parity được giữ cố định; kết quả aggregate là trung bình của Scratch, PyTorch và TensorFlow trên validation.

Bảng 10. Basic so với Improved trên parity validation

Task	Metric chọn	Basic	Improved	Thay đổi	Quyết định
Diabetes	F1 ↑	0.4326	0.4538	+0,0212	Improved
House	RMSE ↓	905.589 USD	902.701 USD	−2.888 USD	Improved
Comments	F1 ↑	0.8669	0.8861	+0,0192	Improved

Improved thắng đúng ba tiêu chí đã định: Diabetes F1 0,4326→0,4538; House RMSE 905.589→902.701 USD; Comments F1 0,8669→0,8861. Mức cải thiện House chỉ khoảng 0,32%, nhỏ hơn rõ so với hai classification task; vì vậy quyết định được mô tả là lựa chọn trong search space hiện tại, không phải bằng chứng rằng CNN sâu hơn luôn tốt hơn.

Kết luận: Improved được khóa cho 6 full-scale runs vì thắng parity validation ở cả ba task. Không dùng full test để quay lại chọn kiến trúc.

10. THẢO LUẬN KẾT QUẢ

Bảng 11. Sáu kết quả full-scale cuối

Task	Framework	Params	Best/ran	Metric test chính	Finding
Diabetes	PyTorch	2.562	4/7	F1 0,4535; AUC 0,8241	Recall cao, precision thấp
Diabetes	TensorFlow	2.562	2/5	F1 0,4524; AUC 0,8219	Nhiều TN hơn, Recall thấp hơn
House	PyTorch	3.249	10/13	RMSE 1.290.996 USD; R^2 0,1737	Heavy-tail chi phối
House	TensorFlow	3.249	10/13	RMSE 1.291.741 USD; R^2 0,1727	Gần PyTorch, vẫn yếu
Comments	PyTorch	322.922	3/6	F1 0,9363; AUC 0,9823	Local token pattern phù hợp
Comments	TensorFlow	322.922	3/6	F1 0,9367; AUC 0,9823	Gần như tương đương

Bảng 12. Tóm tắt training history full-scale (train loss / validation loss)

Task	Framework	Epoch 1	Checkpoint chọn	Epoch cuối	Diễn giải
Diabetes	Pytorch	0.5400 / 0.5203	e4: 0.5086 / 0.5197	0.5061 / 0.5121	validation dao động
Diabetes	Tensorflow	0.5399 / 0.5207	e2: 0.5143 / 0.5230	0.5082 / 0.5114	validation dao động
House	Pytorch	0.6382 / 16.4382	e10: 0.4609 / 235.8504	0.4659 / 1.2112	spike/overflow trung gian
House	Tensorflow	0.6413 / 73.5198	e10: 0.4758 / 1465.2931	0.4679 / 14.6623	spike/overflow trung gian
Comments	Pytorch	0.2139 / 0.1757	e3: 0.1359 / 0.1706	0.0961 / 0.1897	overfit sau vài epoch
Comments	Tensorflow	0.2127 / 0.1744	e3: 0.1352 / 0.1707	0.0954 / 0.1866	overfit sau vài epoch

10.1. Vì sao Comments tốt nhất?

Vị trí kề nhau của token tạo cấu trúc cục bộ thực: các cụm như phủ định + tính từ, n-gram khen/chê hoặc tên sản phẩm + phán đoán có ý nghĩa. Weight sharing cho phép cùng motif được phát hiện ở đầu hay cuối review; GlobalMaxPooling giữ bằng chứng mạnh nhất bất kể vị trí. Diabetes/House không có tính translation tương tự khi đổi vị trí feature.

10.2. Vì sao Diabetes chỉ F1 khoảng 0,45?

Lớp positive chỉ khoảng 13,9%. Class weight nâng Recall lên 0,71–0,73 nhưng làm nhiều negative bị dự đoán positive, kéo Precision xuống 0,33. BRFSS là dữ liệu tabular tự báo cáo với tín hiệu chồng lấn; Conv1D còn áp locality theo thứ tự feature quy ước. AUC ≈0,82 cho thấy ranking hữu ích hơn con số F1 đơn lẻ, song threshold/calibration chưa được tối ưu và không được sửa sau thực nghiệm.

10.3. Vì sao House R^2 thấp?

Chỉ sáu field không nắm bắt location chi tiết, thời điểm, chất lượng và điều kiện thị trường. Price có đuôi dài đến hàng trăm triệu USD; expm1 phóng đại sai số trong log-space thành sai số USD rất lớn. CNN không có locality tự nhiên giữa state, status và numeric fields. Vì vậy $R^2 \approx 0{,}173$ không được che giấu: model chỉ hơn baseline trung bình một phần hạn chế.

10.4. Framework gần nhau đến mức nào?

Diabetes chênh F1 0,0010; House chênh RMSE khoảng 745 USD trên mức 1,29 triệu; Comments chênh F1 0,00042 và AUC gần như bằng nhau. Initial logits parity 10^{-9}–10^{-7} cho thấy topology/weight đã đồng nhất. Sai khác sau train là hệ quả bình thường của thứ tự floating-point, kernel backend và optimizer state, không phải khác biệt bản chất thuật toán.

10.5. Scratch chứng minh điều gì?

Finite-difference, backward parity và tiny-overfit nối trực tiếp công thức convolution với code có thể học. Scratch làm lộ rõ local connectivity, weight sharing, cache argmax, accumulation gradient và Adam bias correction—những chi tiết framework thường che đi. Đây là bằng chứng hiểu CNN mathematics, trong khi 18 parity runs cho thấy implementation thủ công và framework cùng giải một bài toán.

11. HẠN CHẾ

- Chỉ một seed/split và search space Basic–Improved hẹp; không có confidence interval hay seed stability.

- Parity đảm bảo cùng initialization ban đầu, nhưng training đa framework vẫn có round-off/kernel khác nhau; numerical identity sau nhiều epoch không được kỳ vọng.
- Diabetes và House dùng CNN trên thứ tự field do người thiết kế đặt, không có locality tự nhiên; so sánh chưa bao gồm tree/boosting/tabular-specific baselines trong Assignment 04.
- Diabetes BRFSS là dữ liệu quan sát/tự báo cáo; không đánh giá calibration, fairness hay clinical validity.
- House chỉ dùng sáu field, missing/OOV đơn giản và heavy tail rất mạnh. Intermediate expm1 overflow được coi là candidate invalid, không được giải quyết bằng robust loss hoặc domain clipping.
- Comments dùng vocabulary 20.000, MAX_LEN 192 và embedding 16 học từ đầu; không mô hình hóa đầy đủ sarcasm, long-range context hoặc pretrained semantics.
- Timing chỉ thuộc CPU/session cụ thể. Một timing parity đã invalid và bị loại; không có benchmark GPU, memory peak hoặc energy.
- Full master-test bao gồm parity subset; không phải holdout hoàn toàn mới sau parity. Dự án không tune tiếp sau test, nhưng phạm vi suy luận phải ghi rõ.
- Lecture 04 không tồn tại dưới dạng file trong workspace tại thời điểm dựng report; ví dụ convolution được dùng theo thông tin xác thực trong đề bài và được đối chiếu bằng phép tính trực tiếp/source.

12. KẾT LUẬN

Assignment 04 đã hoàn thành một chuỗi kiểm chứng đầy đủ từ toán học đến hệ thống: Conv1D/ReLU/MaxPool/GlobalMax/Dense, loss, backpropagation và Adam được tự cài bằng NumPy; finite-difference và Scratch↔PyTorch/TensorFlow parity đều PASS; 18 parity runs chọn Improved; 6 full-scale runs đánh giá hai framework trên frozen master splits.

Kết quả ủng hộ một kết luận có điều kiện. Improved tốt hơn Basic trong ba task đã thử, nhưng không suy ra mạng sâu hơn luôn tốt. PyTorch và TensorFlow cho metric gần nhau vì cùng toán học; khác biệt chủ yếu nằm ở API, autograd, runtime và khả năng mở rộng. NumPy có giá trị giải thích/kiểm chứng hơn là full-scale production.

Comments phù hợp nhất với Conv1D và đạt F1≈0,937 vì local token patterns có ý nghĩa. Diabetes đạt AUC≈0,82 nhưng F1≈0,45 do imbalance và locality tabular yếu. House chỉ đạt R^2≈0,173 với RMSE≈1,29 triệu USD; kết quả thấp và overflow trung gian được báo cáo nguyên trạng. CNN là một inductive bias: hiệu quả phụ thuộc cấu trúc dữ liệu, không chỉ phụ thuộc framework hay số layer.

Bảng 13. Kết luận cuối và phạm vi sử dụng

Hạng mục	Bằng chứng	Kết luận	Cảnh báo
Scratch correctness	Gradient 10^{-13}–10^{-10}; parity ~10^{-7}	PASS	Không thay thế benchmark full-scale
Kiến trúc	Improved thắng 3/3 parity metric	Chọn Improved	Không khái quát deeper luôn tốt
Diabetes	F1 0,452–0,453; AUC 0,822–0,824	Ranking có tín hiệu	Không chẩn đoán; precision thấp
House	RMSE ≈1,291M; R^2≈0,173	CNN yếu	Heavy-tail; không định giá chuyên nghiệp
Comments	F1≈0,937; AUC≈0,982	Task phù hợp nhất	Overfitting; OOV/truncation
Framework	Initial parity; full metric gần nhau	Cùng mathematics	Timing phụ thuộc môi trường

13. TÀI LIỆU THAM KHẢO

[1] Lecture 04 — Convolutional Neural Networks, môn Intelligent System Development, Dinh Que Tran, Ph.D., Assoc. Prof. Ví dụ x/k được đề bài cung cấp; bản file lecture không có trong workspace khi dựng báo cáo.

[2] CDC BRFSS 2015 — Diabetes Health Indicators dataset; artifact local: data/diabetes/diabetes_binary_health_indicators_BRFSS2015.csv.

[3] USA Real Estate / Realtor dataset; artifact local: data/house_price/realtor-data.zip.csv.

[4] Amazon Review Polarity train/test CSV; artifacts local: data/ecommerce/train.csv và data/ecommerce/test.csv.

[5] PyTorch documentation — torch.nn.Conv1d, MaxPool1d, Embedding, autograd và optim.Adam.

[6] TensorFlow/Keras documentation — Conv1D, Embedding, GradientTape và Adam.

[7] NumPy documentation — sliding_window_view, einsum và add.at.

[8] Ba executed notebooks trong notebooks/assignment04; source trong src/assignment04; kết quả trong results/assignment04; model trong models/assignment04; figure trong figures/assignment04. Đây là nguồn thực nghiệm chính.

Official documentation links: PyTorch Conv1d; TensorFlow Conv1D; NumPy sliding_window_view